Business Requirements Document (BRD)

Project Feature Requirements

Product: Abalyzt Builder + AB Web (Projects)

Generated: 2026-02-28

Document Attribute	Value
Document ID	AB-PRJ-BRD-001
Version	1.0
Status	Draft (derived from implementation)
Prepared By	BA / Systems Analyst
Audience	Product, Engineering, QA, Support
Systems In Scope	`ab-front`, `ab-admin-nextgen`, `ab-api`

Role	Name	Date	Signature
Product Owner			
Engineering Lead			
QA Lead			

Version	Date	Change Description
1.0	2026-02-28	Initial version generated from current codebase behavior (API + Admin + Public web).

Table of Contents

1. Overview
2. Scope
3. Roles And Permissions
4. Domain Model
5. Public Web Requirements (ab-front)
6. Builder/Admin Requirements (ab-admin-nextgen)
7. API Requirements (ab-api)
8. Upload And Media Flows
9. SEO And Caching
10. Community, Votes, Discussions
11. AI Context Integration
12. Non-Functional Requirements
13. Acceptance Criteria
14. Known Gaps And Recommendations

Overview

The Project feature provides hands-on learning projects, downloadable resources, structured guidance, and a submission flow that lets learners share outcomes and optionally publish a submission into Community Solutions. It is managed in the Builder/Admin UI and consumed on the public AB website.

Primary Outcomes

Admins create and publish projects with rich content, guide content, tags, benefits, submission guidance, optional skill mapping, and an AI prompt.

Users browse projects, read content, download resources (auth-gated), submit work, and share to community solutions.

Key Source References

`ab-api/src/modules/project/project.controller.ts` (endpoints)

`ab-front/src/components/projects` (public UX)

`ab-admin-nextgen/src/components/projects/ProjectForm.tsx` (builder UX)

Scope

In Scope

- Project CRUD and publishing (admin).
- Project listing + detail views (public web).
- Rich content (overview) and rich guide content (details) including embedded images.
- Project resources file set (S3-backed) with upload/update/delete flows (admin) and download gating (public web).
- User project submissions: create/update/delete + media upload (auth required).
- Community Solutions integration: publish a submission, list by project, vote, threaded discussion replies.
- AI context integration via `aiPrompt` and rich content extraction.

Out of Scope (For This Document)

- Billing, membership purchase flows, and subscription entitlements beyond the observed gating behaviors.
- Course/lesson “project” lesson type and learning path orchestration.
- Moderation tooling for community content.

Roles And Permissions

Permissions are enforced server-side using CASL (see `ab-api/src/casl/permissions.ts`). The UI layers should reflect these rules.

Role	Projects	Resources	Submissions	Community Solutions / Discussions
guest	List + view only *published* projects.	No downloads (public web explicitly gates downloads behind auth).	No submissions.	Can view discussions (read-only); can view community solutions list (public).
user	List + view published projects (inherits guest).	Can access resource downloads (as implemented by UI gating when authorized).	Manage own project submissions (`Action.Manage` where `userId` matches).	Create discussions + votes (subject to email verification restrictions in CASL).
admin	Create/update/delete, list all, view by ID.	Manage resources + media upload URLs.	Manage submissions (implicitly, since admin can manage all).	Manage all community solution state and discussions.

Email verification constraint: For `user` role, CASL blocks create/edit/delete actions when `emailVerified` is false (`ab-api/src/casl/permissions.ts`). UI should surface a clear message if the API rejects actions for unverified users.

Domain Model

Project

Represents a single learning project. Stored in `projects` and returned as a DTO with computed fields and nested content.

Field	Type	Notes / Rules
`id`	uuid	Generated by API model on insert.
`title`	string	Required.
`slug`	string	Generated from title + short id in API model (`slugify(title) + "-" + nanoid(5)`). Used in public routes `/projects/[slug]` .
`description`	string	Displayed with “read more” truncation on detail view.
`difficultyLevel`	enum	One of: `easy` , `moderate` , `hard` , `very-hard` , `extremely-difficult` .
`duration`	number (minutes)	Optional; displayed only if non-zero on public overview.
`published`	boolean	Defaults false; public listing uses `published=true` .
`deleted`	boolean	Soft-delete. API queries mostly apply `filterDeleted` .
`tags`	string[]	Text array (GIN indexed in DB). Used for filtering and display.
`benefits`	string[]	Shown as “Skills you learn” on public overview. Builder supports ordering (drag/drop).
`submissionGuide`	string[]	Shown on submission tab as “What Makes a Good Submission”. Builder supports ordering (drag/drop).
`skillId`	uuid (optional)	If present, submission creation schedules a background job to award points (see API service behavior).
`aiPrompt`	string (optional)	Additional AI context for the project.
`content`	rich text	`{ id, raw, html }` . Rendered on overview tab.

Field	Type	Notes / Rules
`guide`	rich text	`{ id, raw, html }`. Rendered on details tab.
`resource`	resource bundle	S3-backed metadata with files list. URLs are returned as CDN URLs.
`points`	number	Computed from difficulty (see `ab-api/src/modules/project/utils/calculate-project-points.ts`).
`stats.completed`	number (optional)	Returned for “get by slug” as distinct user submission count.

Project Submission

A user-created submission associated with a project. Stored in `project_submissions`.

Field	Type	Notes / Rules
`id`	uuid	Generated on insert.
`projectId`	uuid	Must reference a published, non-deleted project at submission time.
`userId`	uuid	Set from authenticated user in API service.
`description`	string	Public web uses a Markdown editor and renders markdown.
`links`	string[]	Public web currently supports a single link input (`links[0]`), but the API supports an array.
`images`	string[]	Stored as path keys; API maps them to CDN URLs when returning DTOs.
`isCommunitySolution`	boolean (derived)	Included when listing submissions: true if user has published that submission as a community solution.
`user`	object	Joined user details for display.

Public Web Requirements (ab-front)

Projects List Page

Route: `/projects`.

- Must list only published projects.
- Supports search by title (`q`) with debounce (~300ms).
- Supports multi-select filter on `difficultyLevel`.
- Uses infinite scroll pagination; page index is 0-based in API.
- Shows a loading shimmer while fetching and while fetching next pages.

> **Observed implementation:** `ab-front/src/components/projects/ProjectsPage/ProjectsPage.tsx` issues list queries with `published=true`, `sortBy=createdAt`, `order=desc`.

Project Details Page

Route: `/projects/[slug]`.

- Fetch project by slug (guest accessible) and render: title, description (truncated with read more), and tabs.
- Tabs: Overview, Details, Submission (only when user authorized), Community.
- Overview tab renders rich content (`content.raw`), benefits, and stats (points, completed count, duration, published date, tags).
- Details tab renders guide content (`guide.raw`) and a resources panel (if resources exist).
- Resources panel must be gated: show downloads for authorized users; show register/login CTA for guests.
- If authorized, enable bookmarking of project (`resourceType=project`).

Submissions (User)

Visible only to authorized users on the “Submission” tab.

- Show “Past submissions” list (paginated/infinite); this list is expected to show only the current user’s submissions for the project.
- Submission creation flow includes: markdown description (required), optional link(s), optional images uploaded via presigned URL.

- After creating a submission, optionally publish it as a community solution (default enabled), then refresh the list.
- Each submission card must allow toggling “Share to Community” / “Remove from Community” (community solutions create/delete).

Community Submissions

Visible to all users on the “Community” tab.

- List community solutions for the current project (`type=project`) with pagination (page size 10) and sorting (votes/created/updated).
- Support voting; disable vote actions when user is not authorized or when user owns the submission.
- Support viewing and posting threaded replies via the Discussions system on `resourceType=community_solution` and `resourceId=submissionId`.
- Support deep-linking to a specific submission and optionally a specific discussion via query params (see “Deep Links”).

Deep Links

- Project page should interpret `?discussionId=...` to auto-select the Community tab.
- When `?submissionId=...` is present, the community list should find the page containing that submission and open the replies.

Builder/Admin Requirements (ab-admin-nextgen)

Admin pages use Refine data providers. The Builder must support full authoring of project content, assets, and publishing state.

Project List (Admin)

Route: `/projects` in admin UI.

- Must display a pageable list with columns: Title, Published Status, Created At, Updated At, Difficulty Level, Duration.
- Must support filtering by: query (`q`), published status, deleted status, tags, and (optionally) order number.
- Row click navigates to edit.

> **Observed implementation:** `ab-admin-nextgen/src/pages/projects/index.tsx` drives filters via URL query params.

Create / Edit Project

Routes: `/projects/create`, `/projects/:id/edit`.

Section	Requirements
Basic Info	Title (required), Description (required), Difficulty Level (required), Duration (minutes), Skill selection (recommended), AI Prompt (optional).
Benefits	Maintain a list of "benefits/skills learned" strings. Support add/edit/delete and drag-drop ordering.
Submission Guide	Maintain a list of "what makes a good submission" strings. Support add/edit/delete and drag-drop ordering.
Tags	Maintain a unique list of tag strings. Support add (Enter key), remove.
Rich Content	Project Overview editor (maps to `content.raw/html`) and Project Details/Guide editor (maps to `guide.raw/html`). Must support image upload via presigned URL and store the returned *path* rather than full CDN URL.
Resources	Manage downloadable files for a project (multi-file). Support: select files, upload (presigned URL to S3), attach metadata to project, list, and delete.
Publish	Toggle published/unpublished via dedicated status endpoint. Publish action should be independent from editing content.

Slug authoring: Admin UI currently collects and validates `slug`, but the API create/update DTO omits `slug` and will strip it due to request whitelisting. Treat slug as *system-generated* unless the API is changed to accept it.

API Requirements (ab-api)

All endpoints are under the global prefix `/api` (see `ab-api/src/main.ts`).

Project Endpoints

Method	Path	Auth	Purpose
`GET`	`/api/project`	Guest allowed	List projects with offset pagination and filters.
`GET`	`/api/project/slug/:slug`	Guest allowed	Fetch a published project by slug; includes `stats.completed`.
`GET`	`/api/project/:id`	Bearer required	Fetch by ID (admin usage), includes `mediaCdn`.
`POST`	`/api/project`	Bearer required	Create a new project (admin).
`PATCH`	`/api/project/:id`	Bearer required	Update an existing project (admin).
`PATCH`	`/api/project/:id/status`	Bearer required	Update published status.
`DELETE`	`/api/project/:id`	Bearer required	Soft-delete (sets `deleted=true`).

List Filters (GET /api/project)

Query Param	Type	Behavior
`page`	number	0-based page index; default 0.
`pageSize`	number	1..50; default 20.
`order`	`asc|desc`	Sort order; default asc in DTO, but clients may override.
`sortBy`	`createdAt|updatedAt`	Default createdAt.
`q`	string	Title ILIKE match.

Query Param	Type	Behavior
`published`	boolean	If omitted, returns all (subject to permissions). Guests effectively see only published due to CASL.
`difficultyLevel`	string[]	Filter by difficulty levels.
`tags`	string[]	Matches rows where tags contain all provided values (Postgres array containment).
`orderNumber`	number	Implemented in service as a filter on column `order` (see gaps section; schema mismatch risk).

Resource Management Endpoints (Admin)

Method	Path	Purpose
`POST`	`/api/project/:id/resource/upload`	Get S3 presigned upload URL for a resource file key `projects/:projectId/:filename`.
`PUT`	`/api/project/:id/resource`	Attach/merge a resource file record into project resource metadata.
`DELETE`	`/api/project/:id/resource`	Delete a resource file by `fileKey` (removes from S3 and metadata).

Project Media Upload (Rich Text Images)

Method	Path	Purpose
`POST`	`/api/project/media/presigned-url`	Get presigned URL for images embedded in project rich text content/guide.

Submission Endpoints

Method	Path	Auth	Purpose
`POST`	`/api/project/submissions`	Bearer required	Create a submission. If project has `skillId`, schedule points-awarding job.
`GET`	`/api/project/:projectId/submissions`	Bearer required	List submissions for project, scoped to current user in service logic.
`PUT`	`/api/project/submissions/:submissionId`	Bearer required	Update a submission (permission-scoped).
`DELETE`	`/api/project/submissions/:submissionId`	Bearer required	Delete a submission.
`POST`	`/api/project/submissions/media/presigned-url`	Bearer required	Presigned URL for submission media uploads (stored under `project-submissions/`).

DTO whitelisting: Global validation pipe uses `whitelist: true`. Unrecognized properties sent by clients are stripped (see `ab-api/src/main.ts`).

Request / Response Examples

Examples show shapes used by clients; some fields are optional.

```
{
  "title": "Analyze Customer Churn",
  "description": "Build a churn model and explain drivers.",
  "difficultyLevel": "moderate",
  "duration": 180,
  "tags": ["classification", "business"],
  "benefits": ["Feature engineering", "Model evaluation"],
  "submissionGuide": ["Include notebook", "Explain assumptions"],
  "skillId": "<uuid>",
  "aiPrompt": "You are a professional data analyst...",
  "content": { "raw": { "type": "doc", "content": [] }, "html": "<p>...</p>"
```

```
  "guide": { "raw": { "type": "doc", "content": [] }, "html": "<p>...</p>" }
}
```

Paginated list response:

```
{
  "totalCount": 42,
  "page": 0,
  "totalPages": 3,
  "results": [
    {
      "id": "<uuid>",
      "title": "Analyze Customer Churn",
      "slug": "analyze-customer-churn-a1b2c",
      "description": "...",
      "difficultyLevel": "moderate",
      "duration": 180,
      "published": true,
      "tags": ["classification"],
      "benefits": ["..."],
      "submissionGuide": ["..."],
      "points": 0,
      "createdAt": "2026-02-28T00:00:00.000Z",
      "updatedAt": "2026-02-28T00:00:00.000Z"
    }
  ]
}
```

Error Handling Requirements

- `401 Unauthorized` when Bearer token missing/invalid for protected endpoints.
- `403 Insufficient Permissions` when CASL denies the action.
- `404 Not Found` for missing project/submission/resource where applicable.
- `422 Validation` (wrapped validation errors) when DTO validation fails.

Upload And Media Flows

Admin: Project Resource Upload

1. Admin selects one or more files in Builder.
2. For each file, Builder requests presigned upload URL: `POST /api/project/:id/resource/upload` with `{ filename, contentType, contentMd5 }`.
3. Builder uploads file bytes to returned `presignedUrl` (S3).
4. Builder attaches metadata to project: `PUT /api/project/:id/resource` with `{ filename, path, fileSize }`.
5. Public web receives resource file list with CDN URLs and presents as downloads (authorized only).

Admin: Embedded Images In Rich Text

1. Builder editor requests presigned URL via `POST /api/project/media/presigned-url`.
2. Builder uploads to the returned `presignedUrl`.
3. Builder stores the returned `path` into rich text `raw` / `html` (not the full CDN URL). On subsequent loads, UI can prefix with `mediaCdn`.

User: Submission Image Upload

1. User selects an image in the submission dialog.
2. Client requests presigned URL via `POST /api/project/submissions/media/presigned-url`.
3. Client uploads to S3 using the presigned URL.
4. Client includes returned `path` in `images[]` when calling `POST /api/project/submissions`.
5. API returns DTO where images are transformed into full CDN URLs.

SEO And Caching (Public Web)

- Project detail pages should be statically generated where feasible and revalidated periodically.
- Observed behavior: `/projects/[slug]` uses Next.js static generation with `revalidate: 60` and builds up to 25 published slugs by querying the API.
- When static generation is skipped (env flag), the page should fall back to blocking SSR for requested slugs.

Community, Votes, Discussions

Community Solutions (Project Type)

Community Solutions are a curated/published view of submissions, aggregated by resource (project id) and filtered by type.

Capability	API	Notes
Publish submission to community	`POST /api/community-solutions/:submissionId` body `{ type: "project" }`	Used from the user submission flow ("Publish this to Community").
Remove from community	`DELETE /api/community-solutions/:submissionId`	Used to unshare a previously shared submission.
List community solutions for a project	`GET /api/community-solutions/:resourceId` query includes `type=project`	Public endpoint; supports sorting and pagination.
Vote/unvote	`POST` / `DELETE /api/community-solutions/:submissionId/vote`	Auth required; UI disables voting on own submissions.
Fetch voted submission ids	`POST /api/community-solutions/list/votes`	Used to render vote state efficiently per page of results.

Discussions For Community Solutions

Replies are stored as discussions with `resourceType=community_solution` and `resourceId=submissionId`. Navigation deep links for discussions resolve to project routes with query params `discussionId` and `submissionId`.

> **Observed navigation rule:** When a discussion is tied to a project community solution, API resolves a path like `/projects/:slug?discussionId=...&submissionId=...` (`ab-api/src/modules/discussions/utils/build-navigation-path.ts`).

AI Context Integration

Projects support an optional `aiPrompt` field used as additional context for AI assistant behavior. When AI context is resolved, the API converts rich text HTML to markdown and provides a structured payload.

Context	Resolver Input	Resolver Output (Project)
`project`	`{ context: "project", resourceId, withUnpublished? }`	`{ project: { projectTitle, projectContent, projectGuide, additionalContext } }`

Source: `ab-api/src/modules/ai/utils/context-resolver.ts` includes a project resolver that fetches `[content, guide]`, enforces `published=true` unless `withUnpublished` is set, and turndowns HTML to markdown.

Non-Functional Requirements

Security

- All write endpoints require Bearer auth and must enforce CASL permissions.
- Presigned URLs must require appropriate permissions (resources: `Action.Manage Resource`; submissions: `Action.Manage ProjectSubmission`).
- Soft-deleted projects must not appear in public surfaces.

Performance

- Pagination defaults: `pageSize=20`; max 50; public community list uses 10.
- Public listing uses infinite scroll; must avoid refetch storms during scroll (intersection observer threshold and request de-dupe via query library).

Reliability And Observability

- API logs key actions (create/update/delete/status change/submission create/delete/resources changed).
- Client should handle network failures with clear toasts/messages, and keep UI state consistent (e.g., revert vote state when mutation fails).

Background Jobs (Points / Completion)

- When a submission is created for a project with `skillId`, API schedules a job key `job.project.completed-points` to award completion points.
- Job de-duplication should be scoped to user+project (avoid globally singleton jobs that can collapse multiple users into one execution).

Content Quality

- Rich text content must be valid JSON document schema as expected by the renderer.
- Embedded image paths should be normalized to store relative paths (no CDN prefix in stored data) to keep environments portable.

Acceptance Criteria

Admin / Builder

- Create a project with title, description, difficulty, content, guide, benefits, tags, and submission guide; API returns a project with generated `id` and `slug`.
- Upload at least one resource file; file appears in project resource metadata with a public CDN URL; deleting the file removes it from metadata and S3.
- Publish/unpublish toggles visibility on public web without requiring re-save of other fields.
- Embedded images in rich text successfully upload, render on public web, and remain valid when changing environment/CDN base.

Public Web

- Guest can browse published projects and view project details by slug.
- Guest sees resource lock UI and is guided to register/login to download.
- Authorized user can download resources and create a submission with markdown + optional images/links.
- Sharing a submission to community makes it appear in the Community tab list; unsharing removes it.
- Community list supports sorting and pagination and supports votes and threaded discussions for authorized users.

API

- GET list respects pagination and filters and never returns soft-deleted projects.
- GET by slug only returns published projects and includes `stats.completed` as distinct submitter count.
- Submission creation rejects unpublished/non-existent projects and enforces user permissions.

Known Gaps And Recommendations

Slug mismatch: Admin requires and edits `slug`, but API strips it. Recommendation: either (1) remove slug input from admin and treat as read-only, or (2) update API DTO + model behavior to allow admin-defined slug with uniqueness validation.

Order/orderNumber mismatch: API filter supports `orderNumber` and queries column `order`, but the projects migration shown does not define an `order` column. Recommendation: confirm schema and either add column + DTO field or remove filter and admin UI affordances.

Submissions list scope: `GET /api/project/:projectId/submissions` returns only the current user's submissions (service includes `andWhere ps.user_id = currentUser.id`). If the product expects admins to review all submissions, add an admin-only listing endpoint or a query param to widen scope.

Links array UX: API supports multiple links, but public web UI only exposes `links[0]`. Recommendation: allow multiple link fields or clarify requirement to single link.

Potential job singleton issue: Submission creation schedules the completion points job using `singletonKey` set to the job key string itself. If the job runner treats singleton keys globally, this can suppress awarding for different users/projects. Recommendation: use a scoped singleton key like `job.project.completed-points:<userId>:<projectId>` (or remove singleton semantics and dedupe in a DB constraint).

This requirement file is derived from observed implementation behavior and is intended to serve as a single spec for the Project feature across admin, public web, and API.